4-bit Array Multiplier: RTL to GDSII on SkyWater SKY130

Guru Shwadeep Dornadula (2023112020) · Sudershan Sarraf (2023102015)

Digital VLSI Design assignment · Repository: https://github.com/GuruSwaD/dvd-assignment-

1 Introduction

This report takes a 4-bit × 4-bit unsigned array multiplier from Verilog RTL to a DRC- and LVS-clean GDSII layout with open-source tools on the SkyWater 130 nm process and the `sky130_fd_sc_hd` standard-cell library. Both inputs and the 8-bit product are registered, so the latency is 2 clock cycles and the throughput is one product per cycle. The flow is RTL and gate-level simulation (Icarus Verilog), logic synthesis (Yosys 0.62 with ABC), static timing analysis (OpenSTA 2.7.0, typical corner `tt_025C_1v80`) and physical design with LibreLane 3.0.16 (OpenROAD, Magic, KLayout, Netgen). The tools ran on Windows 11 inside WSL 2 (Ubuntu 24.04) from LibreLane's Nix environment instead of the IIC-OSIC-TOOLS Docker image; the tools and the SKY130 PDK (open_pdks `8afc834`) are the same.

2 Design and verification (Part A)

Design. `full_adder` uses two `assign` statements, $sum = a \oplus b \oplus c_{in}$ and $cout = ab + c_{in}(a \oplus b)$. `mult_array` registers a and b into `a_r` and `b_r` (8 flip-flops with asynchronous active-low reset), forms the 16 partial-product bits $pp_i[j] = a_r[j] \cdot b_r[i]$ and adds them with three rows of four explicitly instantiated `full_adder`s (12 instances), wired exactly as in the connection table of the guide. Row 0 is AND gates only; every later row is a 4-bit ripple-carry adder whose lowest sum is a finished product bit, while the other three sums and the final carry pass down to the next row. The 8-bit result is registered into `p` (8 more flip-flops, 16 in total). The `*` operator is not used in the design.

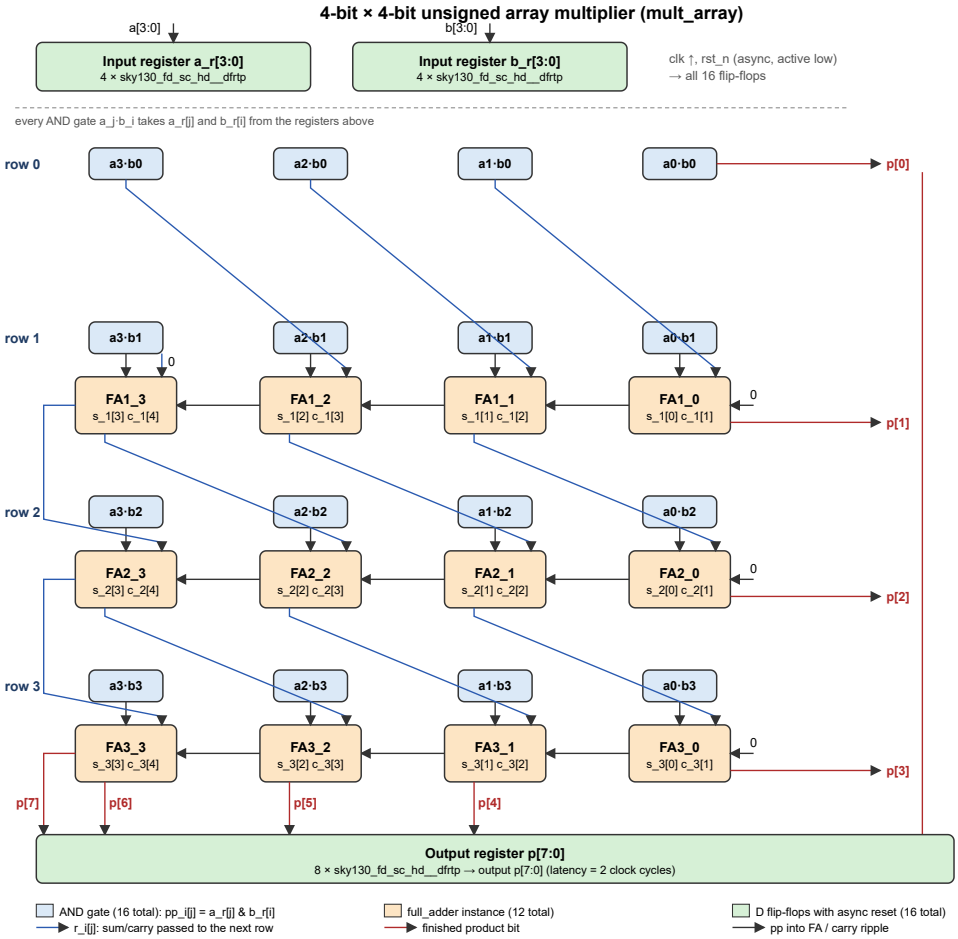


Figure 1: Block diagram of `mult_array`: 16 AND gates (blue), 12 full adders in three ripple-carry rows (orange) and 8 input plus 8 output flip-flops (green). Blue wires carry r_i to the next row; red wires are finished product bits.

Adder architecture. Each of the three adder rows is a 4-bit ripple-carry adder: the carry-out of FAi_j drives the carry-in of FAi_(j+1), and the three upper sums plus the final carry of a row are the addend of the next row. The multiplier is therefore an array of ripple-carry adders, not a carry-save array or a Wallace/Dadda tree. Every

adder is the five-gate full adder of Figure 2: XOR1 forms $a \oplus b$, which is shared by the sum (XOR2) and the carry (AND2 and OR), and AND1 forms $a \cdot b$. The sum costs two XOR delays, and along the ripple chain the carry passes through only AND2 and OR, i.e. two gate delays per bit. Four adders always have one input at 0 (FA1_0, FA2_0 and FA3_0 because $c_i[0] = 0$, and FA1_3 because $r_0[3] = 0$), so they act as half adders ($sum = a \oplus b$, $cout = a \cdot b$); they are still `full_adder` instances because the guide asks for 12. As written, the design has $16 + 12 \times 5 = 76$ gates plus 16 flip-flops.

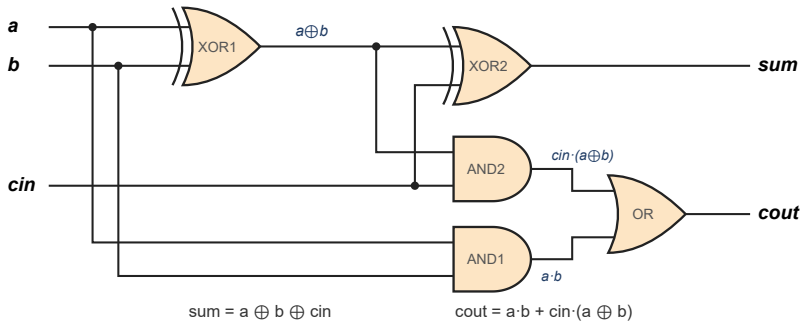


Figure 2: Gate-level implementation of `full_adder` as written in `rtl/full_adder.v`: two XOR, two AND and one OR gate, with $a \oplus b$ shared between sum and carry.

Alternatives. Faster multipliers keep the carries out of the rows (a carry-save array with one fast adder at the end), reduce the partial products in parallel layers (Wallace or Dadda trees, delay $\propto \log N$), halve the number of partial products (Booth encoding), or use carry-lookahead adders in each row. The ripple-carry array was used because the guide specifies this structure; its regularity is also why its delay grows linearly with N (Q6).

Testbench. `tb_mult` generates a 10 ns clock, holds `rst_n` low for the first two cycles and then applies all 256 (a, b) pairs. Each pair is applied on a falling edge; the testbench waits for two rising edges (the 2-cycle latency) and compares p with $a \times b$ in the middle of the following cycle using `!==`, so X or Z values also count as errors. Every mismatch is printed, one summary line ends the run, and the waveform is written to `mult.vcd`.

Table 1: Simulation results.

Test	Result
RTL simulation (`iverilog -g2012`)	`PASS: 256 vectors checked, 0 errors`
Error injection: `sum = a ^ b` in `full_adder.v`	`FAIL: 256 vectors checked, 95 errors`
Gate-level netlist simulation (Part B)	`PASS: 256 vectors checked, 0 errors`

With the broken full adder the first mismatch is a = 3, b = 3 (p = 5, expected 9) at 1060 ns, so the testbench detects real errors; after undoing the change it prints PASS again.

Hand-worked example (Guru Shwadeep Dornadula). Roll number 2023112020 ends in 0, so $A = 0 + 5 = 5$ and $B = 7$. Table 2 follows the multiplication through the rows of the array and Figure 3 shows it in simulation.

```
      0 1 0 1      A = 5
    x 0 1 1 1      B = 7
---------------
      0 1 0 1      pp0 = A AND b0 (b0 = 1)
    0 1 0 1        pp1 = A AND b1 (b1 = 1), shifted 1
  0 1 0 1          pp2 = A AND b2 (b2 = 1), shifted 2
0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
---------------
0 0 1 0 0 0 1 1      P = 35
```

Table 2: Hand multiplication 5×7 (top) and the values inside the array, row by row (bottom; bit vectors MSB first).

Row	pp_i	r_{i-1}	s_i	$c_i[4{:}1]$	Product bits	r_i
0	0101	–	–	–	$p[0] = 1$	0010
1	0101	0010	0111	0000	$p[1] = 1$	0011
2	0101	0011	1000	0111	$p[2] = 0$	0100
3	0000	0100	0100	0000	$p[3] = 0$, $p[7:4] = 0010$	–

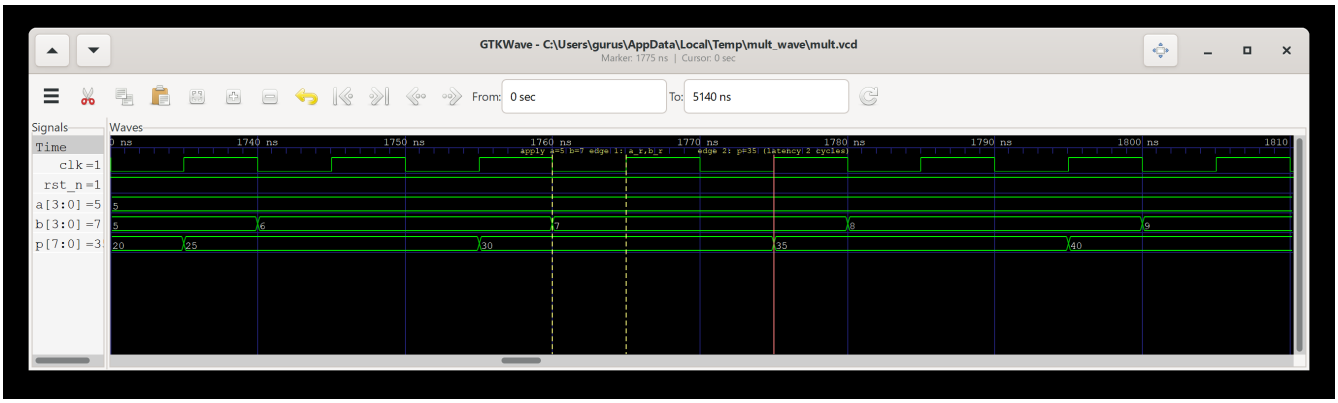


Figure 3: GTKWave, 5×7: $a = 5$ and $b = 7$ are applied at marker A, captured into `a_r`/`b_r` at rising edge 1 (marker B), and the product $p = 35$ appears after rising edge 2 (marker C). Latency = 2 clock cycles = 20 ns at the 10 ns test clock.

Hand-worked example (Sudershan Sarraf). Roll number 2023102015 ends in 5, so $A = 5 + 5 = 10$ and $B = 7$. Table 3 follows the multiplication through the rows of the array and Figure 4 shows it in simulation.

```
      1 0 1 0      A = 10
    x 0 1 1 1      B = 7
---------------
      1 0 1 0      pp0 = A AND b0 (b0 = 1)
    1 0 1 0        pp1 = A AND b1 (b1 = 1), shifted 1
  1 0 1 0          pp2 = A AND b2 (b2 = 1), shifted 2
0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
---------------
0 1 0 0 0 1 1 0      P = 70
```

Table 3: Hand multiplication 10×7 (top) and the values inside the array, row by row (bottom; bit vectors MSB first).

Row	pp_i	r_{i-1}	s_i	$c_i[4{:}1]$	Product bits	r_i
0	1010	–	–	–	$p[0] = 0$	0101
1	1010	0101	1111	0000	$p[1] = 1$	0111
2	1010	0111	0001	1110	$p[2] = 1$	1000
3	0000	1000	1000	0000	$p[3] = 0$, $p[7:4] = 0100$	–

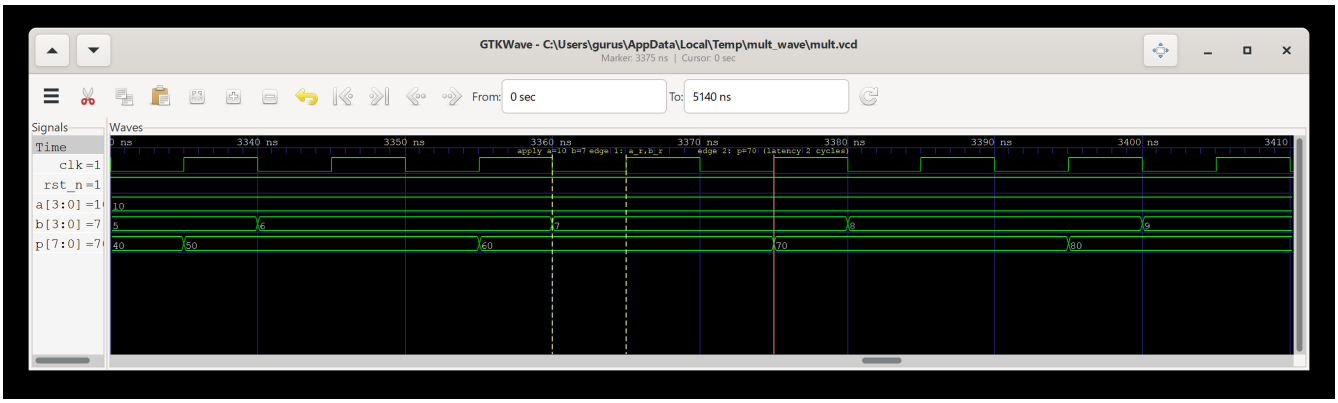


Figure 4: GTKWave, 10×7: $a = 10$ and $b = 7$ are applied at marker A, captured into `a_r`/`b_r` at rising edge 1 (marker B), and the product $p = 70$ appears after rising edge 2 (marker C). Latency = 2 clock cycles = 20 ns at the 10 ns test clock.

3 Synthesis (Part B)

`synth/synth.tcl` runs the seven Yosys steps of the guide (`read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat -liberty`, `write_verilog -noattr`) with the typical-corner Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib`.

Table 4: Part B results (`synth/mult_array_stat.rpt`).

Metric	Value
Total number of cells	70
Number of flip-flops	16 (`dfrtp_1`)
Total cell area	797.014 µm^2
Flip-flop area	$16 \times 25.024 = 400.384$ µm^2
Flip-flop area (% of total)	50.24 %
3 most-used cell types	`dfrtp_1` (16), `nand2_1` (10), `clkinv_1` (6)
Netlist simulation	`PASS: 256 vectors checked, 0 errors`

Table 5: All cell types after synthesis.

Cell	Count	Area (µm^2)
`dfrtp_1`	16	400.384
`xnor3_1`	3	67.565
`maj3_1`	5	50.048
`nand2_1`	10	37.536
`xnor2_1`	3	26.275
`xor3_1`	1	23.773
`clkinv_1`	6	22.522
`a21oi_1`	4	20.019
`and4_1`	2	17.517
`a22oi_1`	2	15.014
`o21ai_0`	3	15.014
`and2_0`	2	12.512
`and3_1`	2	12.512
`a2111o_1`	1	11.261
`nor2_1`	3	11.261
`mux2i_1`	1	10.010
`a22o_1`	1	8.758
`o41ai_1`	1	8.758
`xor2_1`	1	8.758
`nand4_1`	1	6.256
`nor2b_1`	1	6.256
`nor3_1`	1	5.005
Total	70	797.014

How the full adders were mapped. `synth -flatten` removes the `full_adder` boundaries and ABC re-optimises the whole logic cone, so the 76 gates written in the RTL become 54 library cells and the five-gate structure of Figure 2 is not kept literally. Table 6 gives, for every adder, the cell that computes its sum and its carry; it was found by simulating the netlist for all 256 inputs and matching each net against the RTL signals. 4 adders (FA2_1, FA3_1, FA3_2, FA3_3) became the ideal one-cell-per-output form: an XOR3/XNOR3 for the sum and a MAJ3 for the carry, since the majority function $ab + b\,c_{in} + a\,c_{in}$ equals $ab + c_{in}(a \oplus b)$. The half adders became simple gates. In row 1, whose inputs are all partial products, ABC merged the AND gates with the adder logic into AND-OR-INVERT and multiplexer cells (`a21oi`, `and4`, `mux2i`); in 2 adders (FA1_1, FA1_2) one output no longer exists as a single wire.

Table 6: Cell computing each full adder's sum and carry in the synthesized netlist.

Adder	Acts as	Sum cell	Carry cell
FA1_0	half	`nor2`	`and4`
FA1_1	full	*merged*	`a21oi`
FA1_2	full	`xor2`	*merged*
FA1_3	half	`mux2i`	`a21oi`
FA2_0	half	`nor2b`	`nor3`
FA2_1	full	`xnor3`	`maj3`
FA2_2	full	`xnor2`	`maj3`
FA2_3	full	`xnor2`	`o21ai`
FA3_0	half	`nor2`	`and3`
FA3_1	full	`xnor3`	`maj3`
FA3_2	full	`xnor3`	`maj3`
FA3_3	full	`xor3`	`maj3`

4 Timing (Part C)

`sta/sta.tcl` reads the Liberty file and the netlist, defines `clk`, sets 1.0 ns input and output delays, prints the worst setup slack and saves the critical-path report. With $T = 10$ ns the worst slack is 7.017 ns, so $T_{min} = 10 - 7.017 = 2.983$ ns and $F_{max} = 1000/T_{min} = 335.2$ MHz. Re-running with $T = 2.98$ ns gives a slack of -0.003 ns, zero to within the 0.01 ns rounding.

Table 7: Part C results (typical corner).

Metric	Value
Worst slack, $T = 10$ ns	7.017 ns
T_{min}	2.983 ns
F_{max}	335.2 MHz
Slack at $T = 2.98$ ns	−0.003 ns
Start point	`_109_` = `a_r[1]` register (CLK)
End point	`_107_` = `p[7]` register (D)
Cells on path	8 logic cells + 2 flip-flops
Latency $= 2T_{min}$	5.966 ns
Throughput	335.2 million results/s

Table 8: Critical path at $T = 10$ ns (`sta/mult_array_timing.rpt`). The "Computes" column was verified by simulating the netlist for all 256 inputs and matching each net against the RTL signals.

#	Inst.	Cell	Incr	Arrival	Computes
0	`_109_`	`dfrtp_1`	0.396	0.396	launch flip-flop `a_r[1]` (clock-to-Q)
1	`_054_`	`a22o_1`	0.248	0.644	$a_1b_1 + a_2b_0$: AND gates feeding FA1_1
2	`_055_`	`and3_1`	0.181	0.824	FA1_1 sum logic (merged with FA2_0)
3	`_058_`	`nor3_1`	0.276	1.101	$= c_2[1]$, carry of FA2_0
4	`_070_`	`xnor3_1`	0.453	1.554	$= s_2[1]$, sum of FA2_1
5	`_071_`	`and3_1`	0.209	1.763	$= c_3[1]$, carry of FA3_0
6	`_085_`	`maj3_1`	0.361	2.124	$= c_3[2]$, carry of FA3_1
7	`_097_`	`maj3_1`	0.379	2.503	$= c_3[3]$, carry of FA3_2
8	`_098_`	`maj3_1`	0.349	2.852	$= c_3[4]$, carry of FA3_3
	`_107_`	`dfrtp_1`	-0.131		capture flip-flop `p[7]` (setup)

5 Physical design (Part D)

`pnr/config.json` sets `DESIGN_NAME`, `VERILOG_FILES`, `CLOCK_PORT`, `FP_SIZING = "absolute"` and a fixed 150 µm × 150 µm `DIE_AREA`. The first clock period was $1.2\,T_{min} = 3.58$ ns, rounded up to 4.0 ns. LibreLane signs off timing in nine PVT corners and the top-level `timing__setup__ws` in `metrics.json` is the worst of them, the slow `ss_100C_1v60` corner. Following the guide, the period was raised by 0.5 ns per run until that slack was ≥ 0 (Table 9). At 5.5 and 6.0 ns the slack did not improve by the full 0.5 ns: LibreLane's synthesis and resizer are timing-driven and pick smaller, slower cells when the period is relaxed, so the same `a_r` → `p[7]` carry path became longer.

Table 9: Every LibreLane run (also recorded in `README.md`). Slack in ns.

Run	Folder	Period (ns)	Slack, worst	Slack, tt	Setup vio.	DRC	LVS	Action
1	`RUN_2026-10-23_21-30-54`	4.0	-1.254	+1.167	15	0	0	+0.5 ns
2	`RUN_2026-10-23_21-33-00`	4.5	-0.754	+1.667	12	0	0	+0.5 ns
3	`RUN_2026-10-23_21-34-21`	5.0	-0.254	+2.167	9	0	0	+0.5 ns
4	`RUN_2026-10-23_21-35-43`	5.5	-0.009	+2.595	1	0	0	+0.5 ns
5	`RUN_2026-10-23_21-37-01`	6.0	-0.001	+2.929	1	0	0	+0.5 ns
6	`RUN_2026-10-23_21-38-16`	6.5	+0.499	+3.429	0	0	0	timing met (final)

Table 10: Part D results (`pnr/metrics.json`, final run).

Metric	Value
`CLOCK_PERIOD` used	6.5 ns
Number of runs needed	6
Core area	17,759.5 µm^2 (die 22,500 µm^2)
Cells after synthesis (Part B)	70
Cells after place and route	4822 (336 without fill/decap)
Kinds of cells added	decap 4214, fill 272, tap 245, buffers 20, clock buffers 3
Wirelength	2,612 µm
Setup slack	0.499 ns worst corner (tt: 3.429 ns)
Post-route F_{max}	166.6 MHz worst corner (tt: 325.6 MHz)
F_{max} from Part C	335.2 MHz
DRC errors	0 (Magic 0, KLayout 0, routing 0)
LVS errors	0

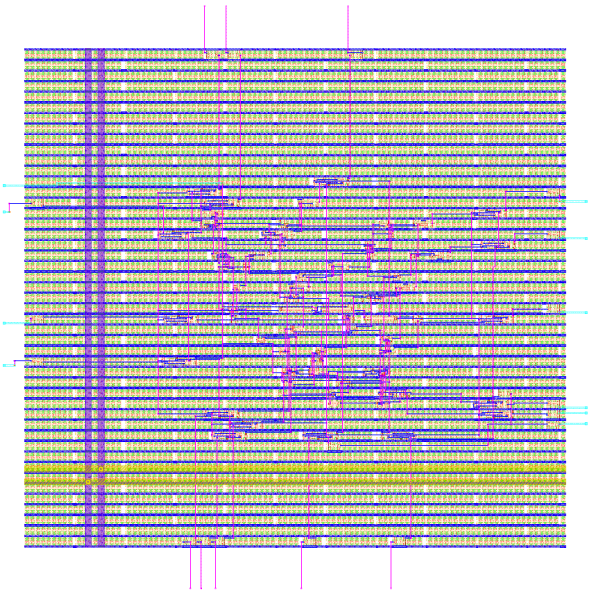


Figure 5: Final layout, whole 150 µm × 150 µm die (KLayout).

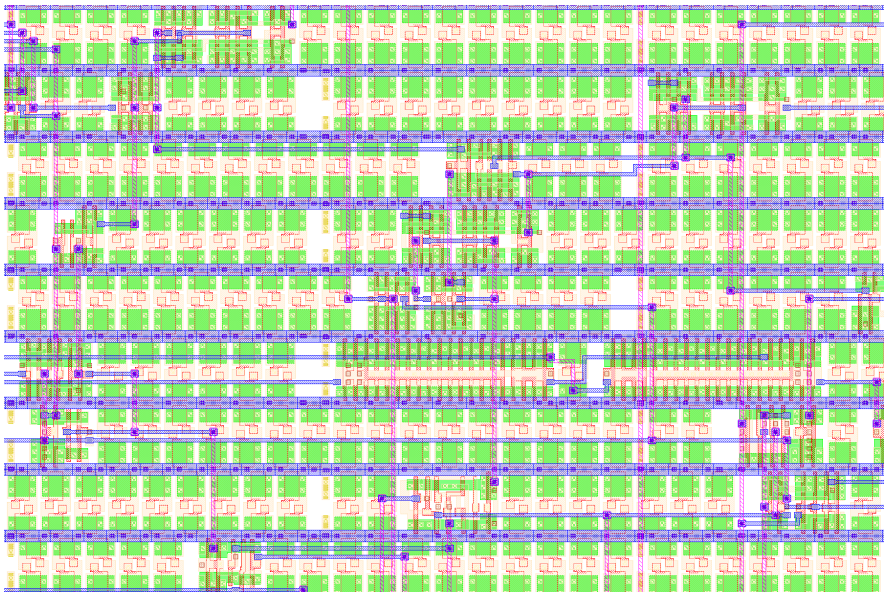


Figure 6: Zoomed layout: standard-cell rows between horizontal met1 power rails, with routed met1/met2 wires and vias; the repeated cells are decap and fill.

The flow also reports 11 max-slew warnings, all in the slow (ss) corners and all on the asynchronous reset net: one fan-out buffer drives ten `RESET_B` pins with a slew of 0.822 ns against a 0.75 ns limit. The reset is asynchronous and quasi-static, so this does not affect timing or function, and it does not appear in the typical or fast corners.

6 Answers to questions

Q1. Synthesis produced 70 cells, of which 16 are flip-flops, so 54 cells implement the logic, not $16 + 12 = 28$. A full adder is not a single cell here (Table 6): ABC maps each sum to an XOR3/XNOR3 (or an XOR2 chain) and each carry to a MAJ3 or an AOI/OAI combination, so one adder needs about two cells, and polarity fixing adds inverters (6 `clkinv`). In the other direction, ABC removes and merges logic because the design is flattened: the zero carry-ins ($c_i[0] = 0$) and $r_0[3] = 0$ turn four of the adders into half adders, and many AND gates are absorbed into complex cells such as `a22o`, `a21oi` and `a2111o`. The cell count is set by the library and by logic optimisation, not by the number of RTL modules.

Q2. The critical path starts at the clock pin of the `a_r[1]` input flip-flop and ends at the D pin of the `p[7]` output flip-flop. It has 8 logic cells (Table 8); data arrive at 2.852 ns, including 0.396 ns clock-to-Q, and the setup time is 0.131 ns. Simulating the netlist shows that the path passes through 7 full adders (FA1_1, FA2_0, FA2_1, FA3_0, FA3_1, FA3_2, FA3_3); the one remaining cell is the pair of AND gates that feed FA1_1. This is close to the estimate of $2N = 8$ full-adder delays. It is one fewer because the longest path enters row 2 at bit 0 instead of crossing every adder of row 1, and it has fewer cells than adders would suggest because single SKY130 cells (MAJ3 for a carry, XNOR3 for a sum) implement a whole full-adder output. On average one full-adder delay is $(2.852 - 0.396)/7 \approx 0.351$ ns.

Q3. Flip-flops take 400.384 µm^2 of 797.014 µm^2, i.e. 50.24 % of the cell area. One `dfrtp_1` is 25.024 µm^2, while a typical logic gate is 3.75–10 µm^2, because a flip-flop with asynchronous reset contains a master and a slave latch, clock inverters and reset logic. The design has 16 flip-flops but only 54 small logic cells, so the registers dominate. Logic grows as N^2 (N^2 AND gates and $N(N-1)$ full adders) while the registers grow only as $4N$, so the flip-flop share falls for wider multipliers.

Q4. Part C used ideal wires, an ideal clock and only the typical corner. After routing, every net has extracted resistance and capacitance (total wirelength 2,612 µm), the clock tree adds insertion delay and skew, LibreLane adds a 0.25 ns clock uncertainty, and timing is signed off in nine corners, including slow silicon at 100 °C and 1.60 V. In the worst corner the slack is 0.499 ns at 6.5 ns, so $F_{max} = 1000/(6.5 - 0.499) = 166.6$ MHz, well below 335.2 MHz from Part C. Even in the same tt corner the post-route value, 325.6 MHz, is lower, because of the wires, the clock uncertainty and the smaller cells that timing-driven synthesis chose for the relaxed period. The flow added: *tap* cells (245) that tie the n-well and substrate to the supplies to prevent latch-up; *decap* cells (4214) that store charge between VPWR and VGND to reduce supply noise; *fill* cells (272) that close the gaps in each row so the wells and power rails are continuous; *buffers* (20: 9 on the inputs and 8 on the outputs to isolate the ports, 2 to split the high-fan-out reset net, 1 delay cell for hold fixing); and *clock buffers* (3) from clock-tree synthesis, so all 16 flip-flops receive the clock with small skew.

Q5. DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths, spacings, enclosures, overlaps and densities on every layer. LVS (layout versus schematic) extracts the

transistors and their connections from the layout and checks that they form the same circuit as the gate-level netlist (same devices, nets and pins). A DRC error means the chip may not be manufactured correctly (shorts, opens, low yield); an LVS error means the manufactured chip is not the designed circuit. Both must pass, as they do here (DRC 0, LVS 0), because either kind of error makes the silicon useless and a mask set is far too expensive to risk.

Q6. For $N = 8$: full adders $N(N-1) = 8 \times 7 = 56$ instead of 12 (4.7×), and AND gates $N^2 = 64$ instead of 16 (4×). The critical path becomes about $2N = 16$ full-adder delays instead of 8 (2×). With the measured 0.351 ns per full adder, $T_{min} \approx 16 \times 0.351 + 0.396 + 0.131 \approx 6.14$ ns, so $F_{max} \approx 163$ MHz compared with 335.2 MHz for $N = 4$. Area grows quadratically with N, delay only linearly.

7 Conclusion

The multiplier passes all 256 vectors at RTL and gate level, synthesises to 70 cells (797.014 µm^2, about half of it flip-flops), reaches $T_{min} = 2.983$ ns (335.2 MHz) in typical-corner STA, and is placed and routed at 6.5 ns with 0 DRC errors, 0 LVS errors and non-negative setup slack in all nine corners. The results match the structure of the circuit: about two cells per full adder, a critical path through 7 full adders ending in the last ripple-carry row, and a layout whose cell count is dominated by fill, decap and tap cells rather than logic.

AI tool disclosure. Claude (Anthropic) was used to help set up the toolchain, write the Verilog, Tcl and JSON files, run the flow and draft this report. We have reviewed every file and can explain each line.